Java 程序设计

授课人：何毅
办公室：1909

CITY INSTITUTE
城市学院

第10讲 内容向导

10.1 GUI概述

10.2 容器和布局

10.1 GUI概述

- 使用框架、面板和简单用户界面GUI组件，分AWT和Swing

- 布局管理器

  使用FlowLayout, GridLayout, BorderLayout管理器

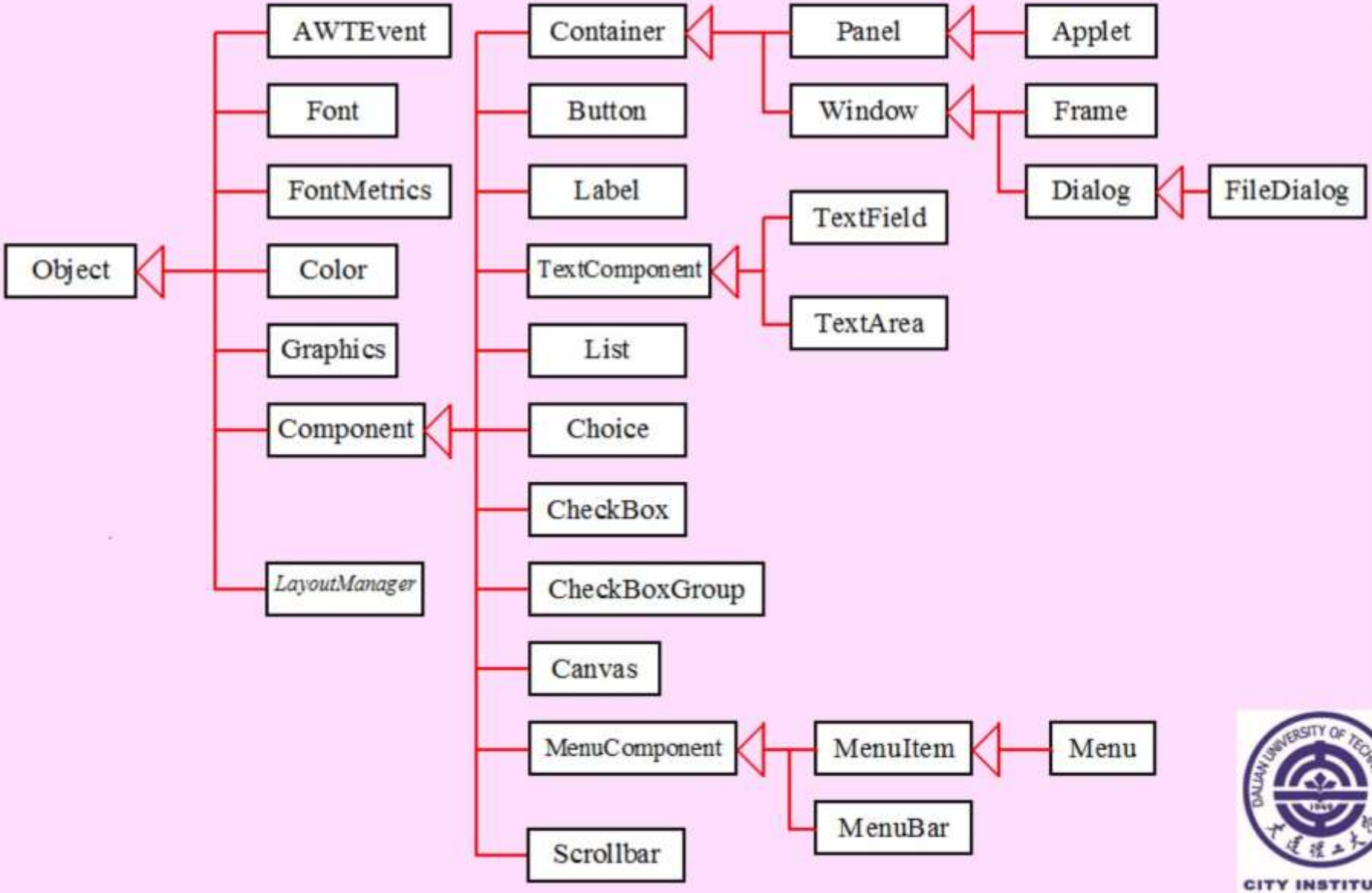
- 在面板上绘制组件

  paintComponent方法熟悉 Colors, Fonts, and Font Metrics类

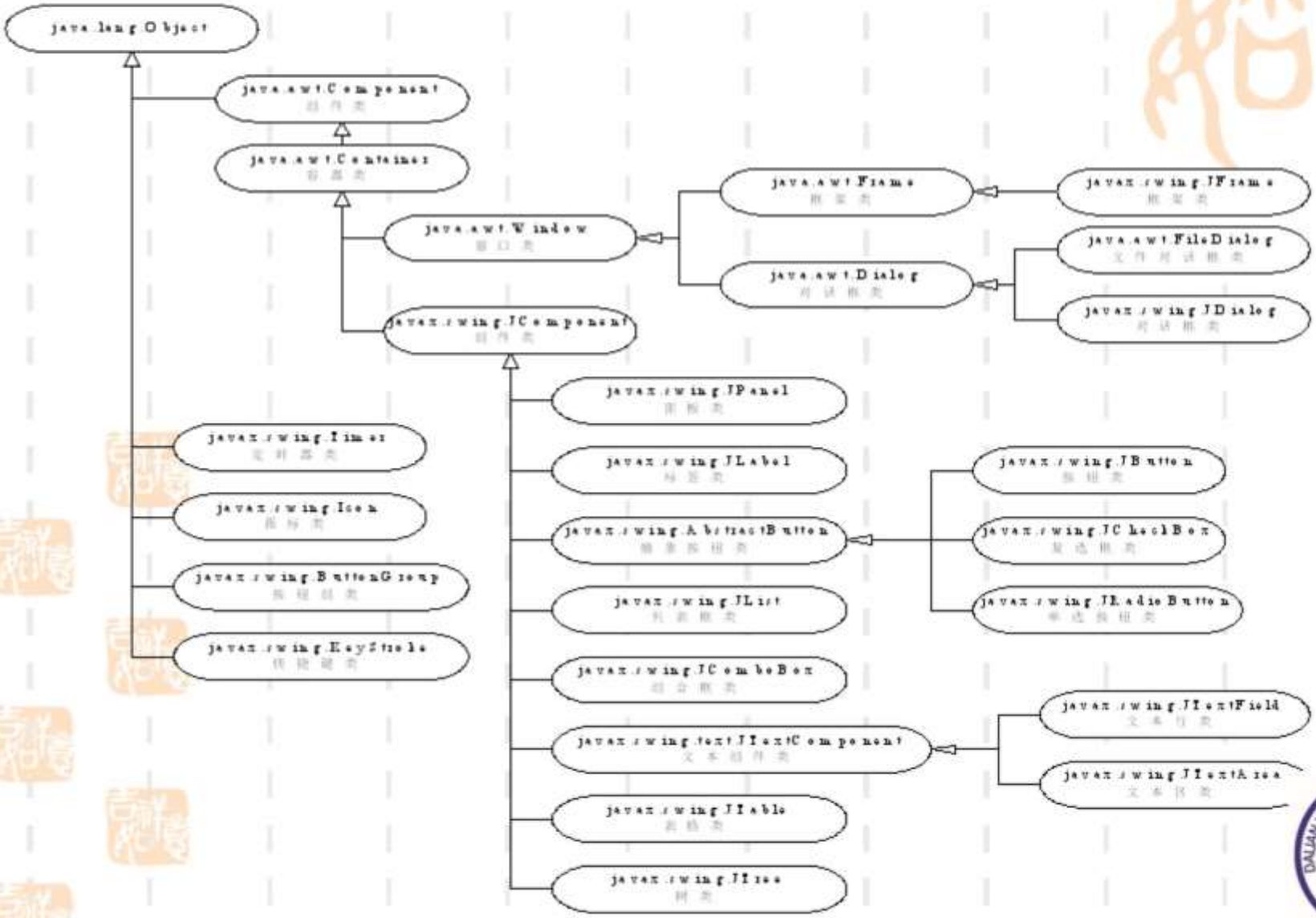
- 理解事件驱动程序设计的概念

事件源，监听器和监听接口 Listener Interface

AWT

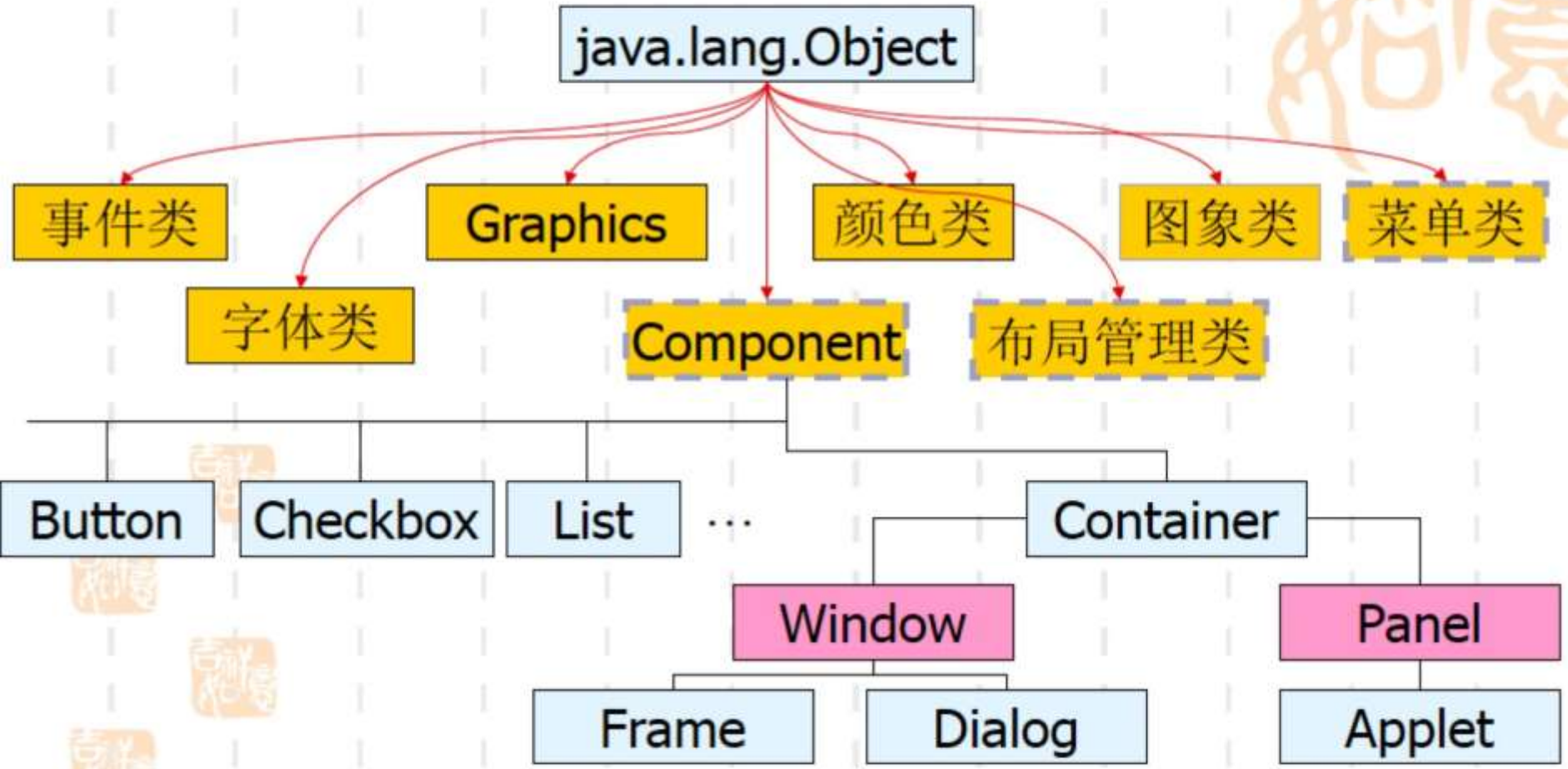


Swing



CITY INSTITUTE
城市学院

lang



JComponent类

常见组件：Button, Checkbox, CheckboxGroup, Choice, Label, List, Canvas, Scrollbar等。

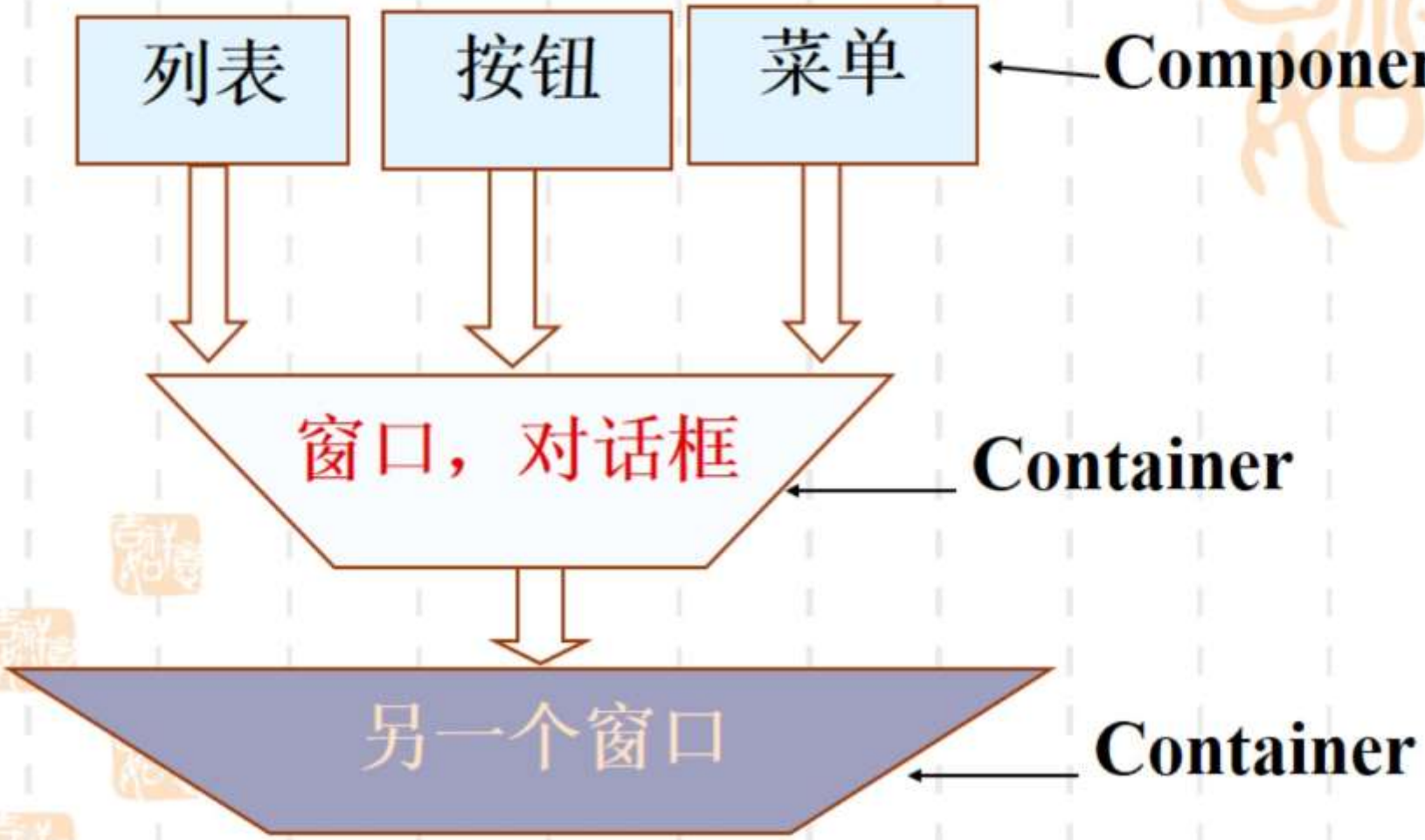
Container类

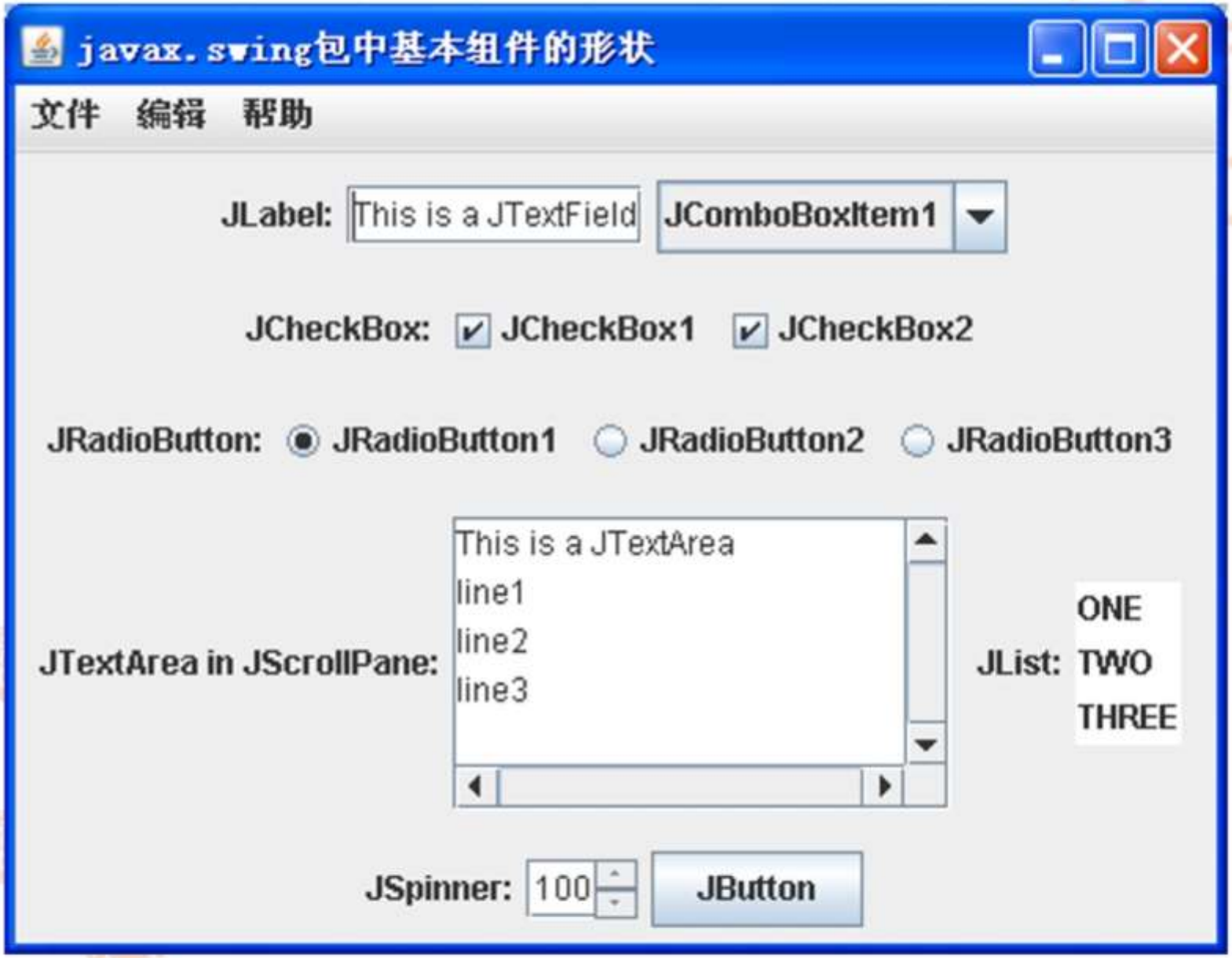
(1)用来表示各种GUI组件的容器

add()　　添加一个组件

remove()删除一个组件

(2)常见Container类有：Frame, Panel, Applet等。



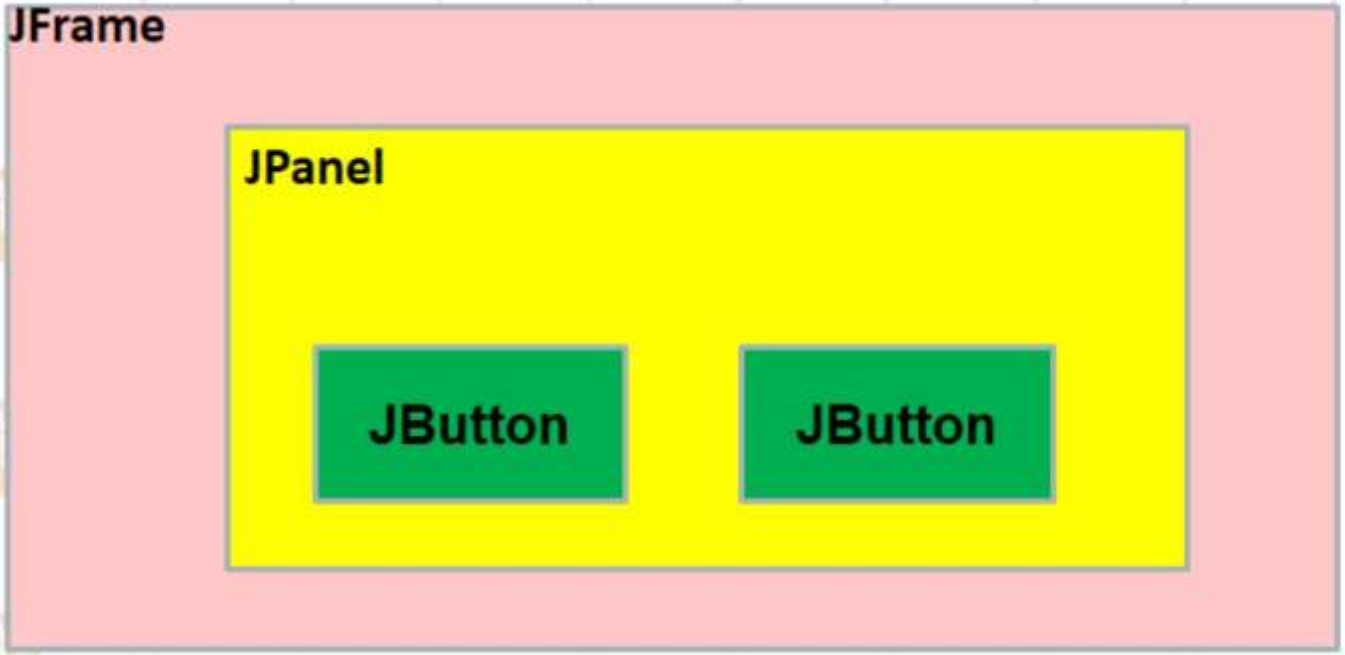


10.2 容器和布局

- 1.顶层容器
  - Swing顶层容器包括JFrame、JDialog和JApplet三种。
  - 所有顶层类都提供获取和设置内容窗格的方法：
    - Container getContentPane()
    - setContentPane(Container contentPane)

2.非顶层容器

- 引入非顶层容器的目的主要是为了更好地管理和布置组件。
- 举例：



- **Container类的常用方法**
  - public Component add(Component comp)
  - public void remove(Component comp)
  - public void setFont(Font f)
  - public void setLayout(LayoutManager mgr)

3. 框架JFrame类

框架是一个不被其它窗体所包含的独立的窗体，是在java中容纳其它用户接口组件的基本单位。

- JFrame类是用来创建一个窗体的。

创建JFrame对象

public JFrame()

声明并创建一个没有标题的JFrame对象。

public JFrame(String title)

声明并创建一个指定标题的JFrame对象。

JFrame类包含的主要方法

- public JFrame() throws HeadlessException
- public JFrame(String title) throws HeadlessException
- public Container getContentPane()
- public void setDefaultCloseOperation(int operation)
- public void setLayout(LayoutManager manager)

一个简单的框架

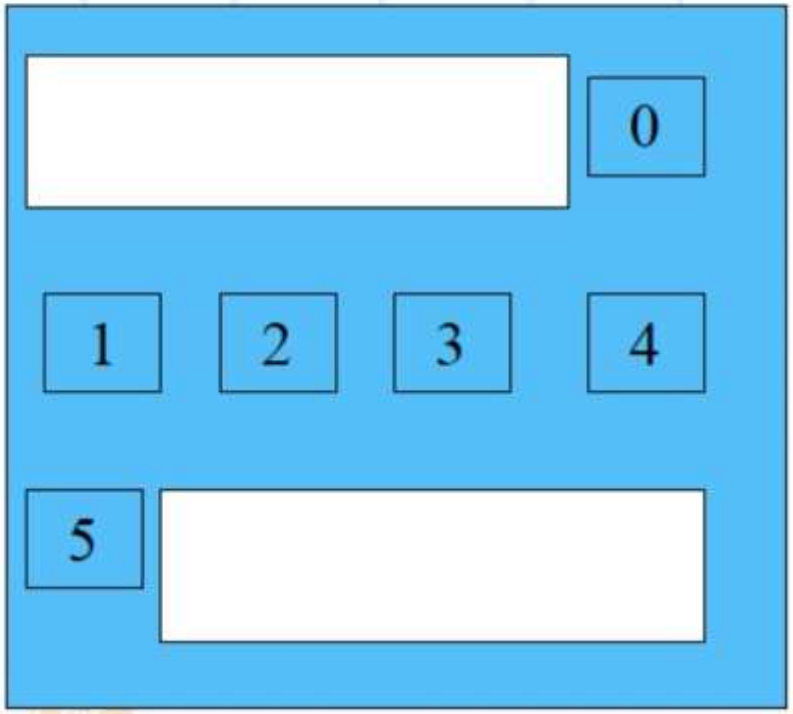
4. 布局管理

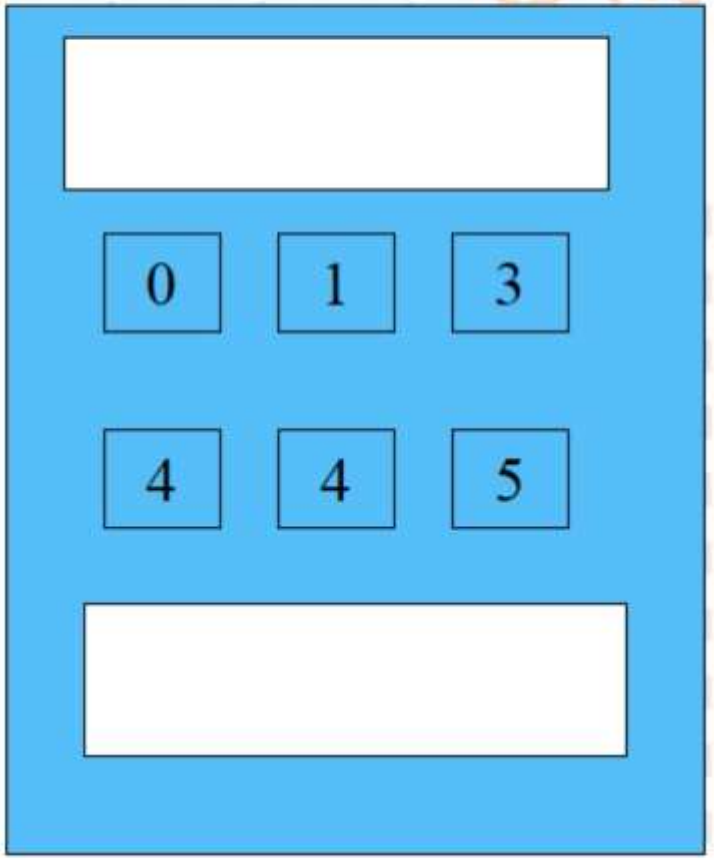
- Java的布局管理器提供了一层抽象，自动把用户界面映射到所有的窗口系统

- Java的GUI组件放在容器中放置。这些GUI组件由容器的布局管理器来安排位置。

- 布局管理器设置在容器中，设置的语法如下：

```
container.setLayout(new
                    SpecificLayou());
```





布局管理器类

(1)管理各种组件在容器中的放置状态
(2)标准的布局管理器类

FlowLayout

BorderLayout

GridLayout

FlowLayout

最简单的布局管理器。

它按添加组件的顺序由左到右将组件排列在容器中，一行排满后再排新的一行。

三个构造方法：

```
public FlowLayout(int align,int hGap,int vGap)
```

对齐方式有：FlowLayout.LEFT 、FlowLayout.RIGHT、FlowLayout.CENTER

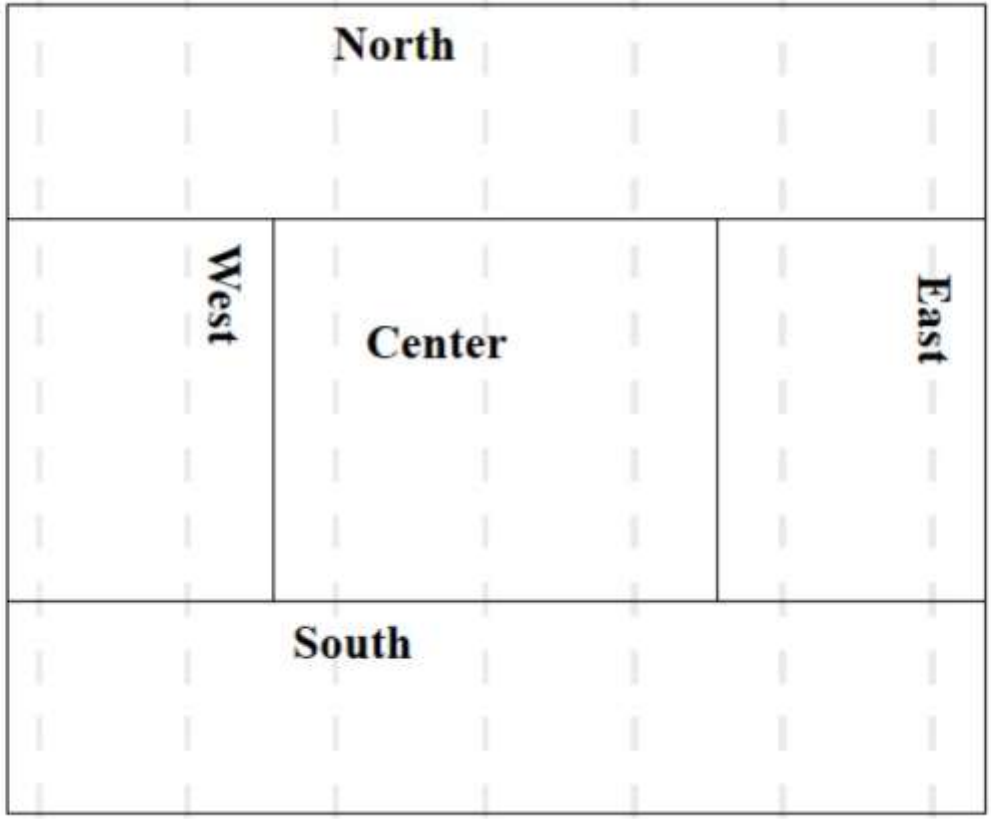
```
public FlowLayout(int align)
public FlowLayout()
```

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
 public class test extends JFrame {
 JButton btn1=new JButton("1"),btn2=new JButton("2");
 JButton btn3=new JButton("3"),btn4=new JButton("4");

public test(){
this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
this.setLayout(new FlowLayout(FlowLayout.LEFT,10,10));
      add(btn1);add(btn2);add(btn3);add(btn4);

      this.setSize(200,100);
      this.setVisible(true);
   }
   public static void main(String[] args){
         new test(); }}
```

BorderLayout

分为东、西、南、北和中五个区域

Frame的缺省布局管理器



两个构造方法：

```
public BorderLayout(int hGap,int vGap)
public BorderLayout()
```

组件根据它们最合适的尺寸和在容器中的位置来放置。南、北组件可以水平扩展，东、西组件能够竖直拉伸，中央组件既可以水平也可以竖直扩展以填满空白空间。

JFrame的内容窗格的默认布局管理器是BorderLayout。

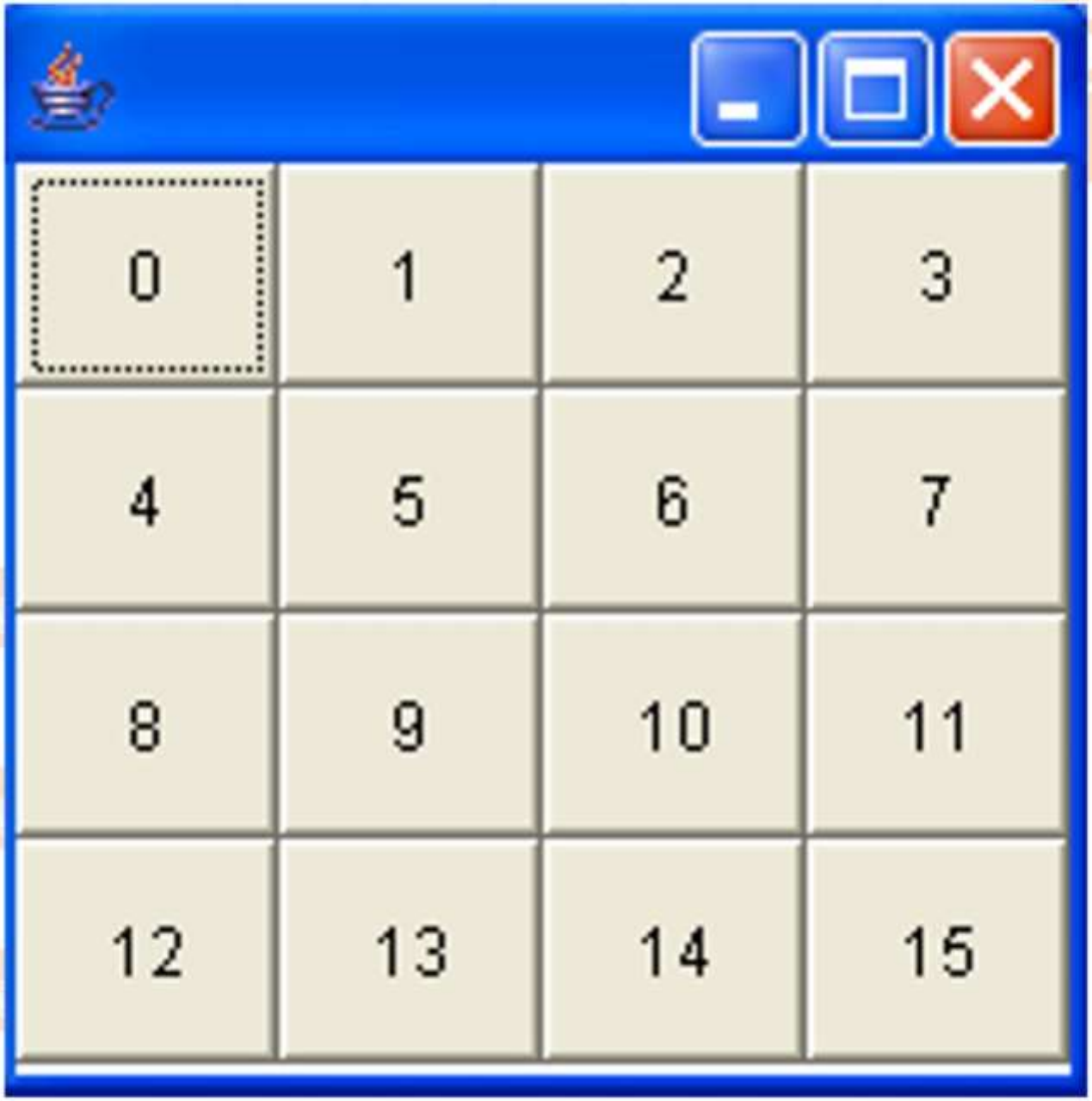
```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
public class test extends JFrame {
   JButton btn1=new JButton("北 "),btn2=new JButton("南 ");
   JButton btn3=new JButton("东 "),btn4=new JButton("西 ");
   public test(){
     this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
      this.setLayout(new BorderLayout(10,10)) ;

add(BorderLayout.NORTH,btn1);add(BorderLayout.SOUTH,btn2);

add(BorderLayout.EAST,btn3);add(BorderLayout.WEST,btn4);
      this.setSize(200,150);
      this.setVisible(true);
   }
   public static void main(String[] args){
         new test(); }  }
```

GridLayout

根据构造方法定义的行数和列数以网格（矩阵）的形式排列组件，组件按添加的顺序从左到右排列。

三个构造方法：

```
public GridLayout(int rows, int columns, int hGap, int vGap)

public GridLayout(int rows, int columns)

public GridLayout()
```



```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
public class test extends JFrame {
   JButton bx[]=new JButton[16];
   int i;
   public test(){
      for(i=0;i<16;i++)
        bx[i]=new JButton(Integer.toString(i));
       this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
       this.setLayout(new GridLayout(4,4));
       for(i=0;i<16;i++)
         add(bx[i]);
       this.setSize(200,200);
       this.setVisible(true);    }
    public static void main(String[] args){
         new test(); }}
```

- 注意：
  - 布局管理器方法：

    setLayout(null)

    setLocation()

    setSize()

    setBounds()

5. 面板组件

非顶层容器，可以将其他控件放在JPanel中以组织一个子界面。通过嵌套使用JPanel，可以搭建出复杂美观的界面。

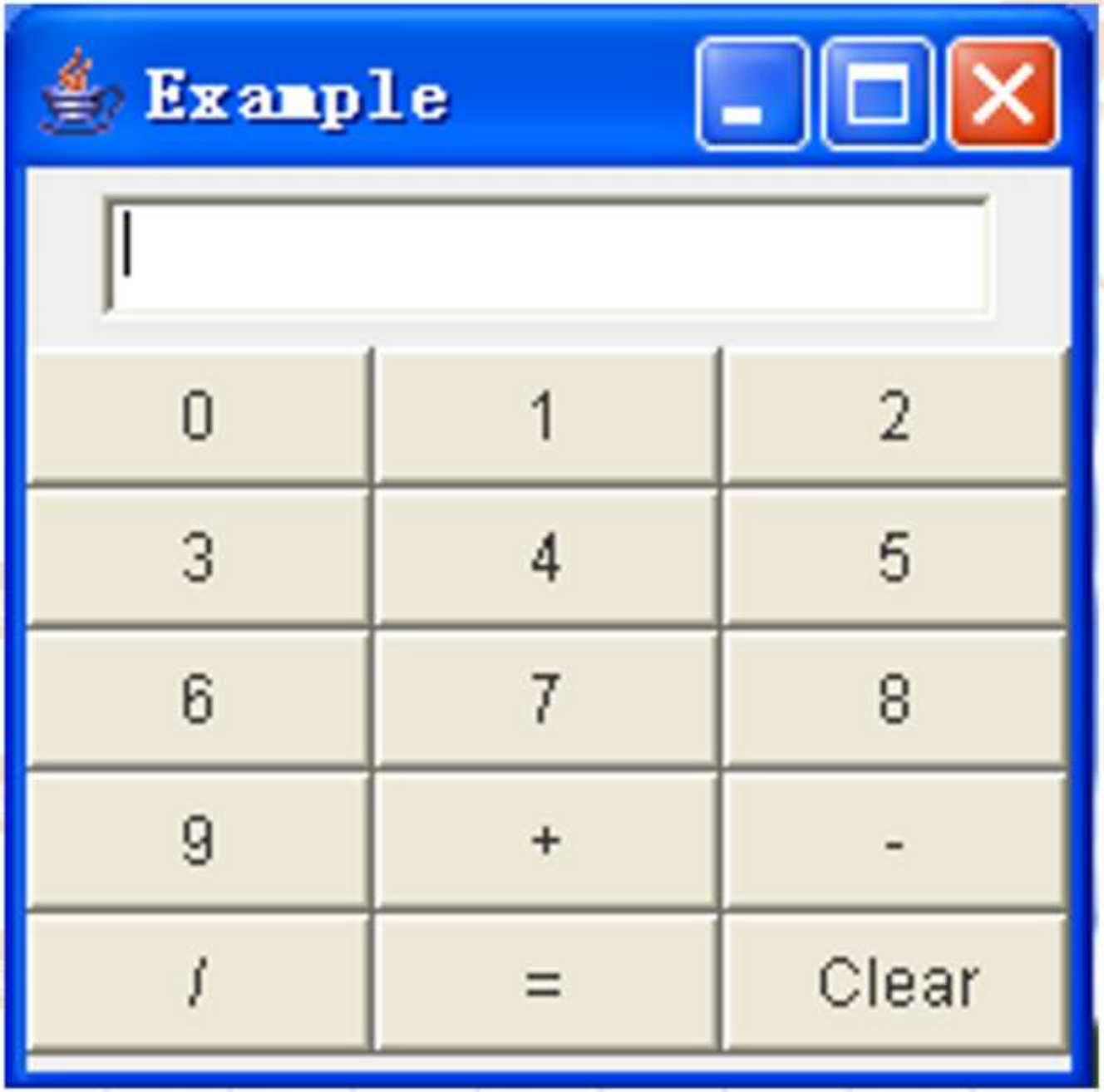
创建方法：

```
JPanel p1=new JPanel()
```

例如：

```
JFrame f=new JFrame()
    f.add(p1);
JButton b=new JButton()
    p1.add(b);
```

- 面板(Panels)是分组放置用户界面组件的更小的容器。
- 通常将组件放置在panel中，将panel放置在框架中，也可在一个面板中再放置面板。
- 面板的Swing版是JPanel。JPanel类的构造方法是JPanel()。默认情况下，JPanel使用FlowLayout。
- 注意：给JFrame添加组件，实际添加到了JFrame的内容窗格中。给面板添加组件，直接添加到了面板中。



```
import java.awt.*;
import javax.swing.*;
public class test {
    String
s[ ]={"0","1","2","3","4","5","6","7","8","9",
                "+","-","/","=","Clear"};
    JFrame f;          JButton b;
    JTextField t;     Panel p1,p2;
    public void display( )
    {       f=new JFrame("Example");
            p1=new Panel();
            t=new JTextField(20);
            p1.add(t);
            p2=new Panel(new GridLayout(5,3))
```

```
        for(int i=0;i<=14;i++)
            p2.add(new JButton(s[i]));
              f.add(p1,"North");
              f.add(p2,"Center");
              f.setSize(200,200);
              f.setVisible(true);

f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
}
  public static void main(String args[ ]){
          new test().display();     }
           }
```

复习与预习

复习

1.JFrame窗体应用

2.布局应用

预习

1.组件应用